

Dental CRM (den_crm) Backend Architecture

Comprehensive Technical Documentation & Operational Learning Guide for Engineering Teams

FASTAPI 0.110+

SUPABASE POSTGRESQL

REDIS 7 / FALLBACK

DOCKER COMPOSE & NGINX

OPENAI / GEMINI SDK

API V1.0.0

Document Purpose: This guide provides an end-to-end operational blueprint of the den_crm backend system. It maps clean architectural patterns, 16 key production features, concrete file locations, request lifecycles, and a non-technical mental model designed for rapid technical onboarding.

1. System Architecture & Tech Stack Matrix

The den_crm backend is a high-performance, multi-tenant RESTful web service designed to manage multi-clinic dental operations, patient acquisition, appointment workflows, revenue accounting, and automated AI leads summarization.

Core Web Framework

FastAPI

Provides fully asynchronous request routing, auto-generated OpenAPI documentation, strict Pydantic v2 data validation, and dependency injection for security/auth.

Database & RLS

Supabase PostgreSQL

Primary relational store for CRM entities. Uses PostgREST Python client for application queries, supplemented by native Row Level Security (RLS) policies.

In-Memory & Cache Store

Redis 7 + Fallback

Handles active JWT token blacklisting, SlowAPI rate limiting, financial idempotency locks, report caching, and webhook deduplication with in-memory fallback.

Reverse Proxy & Gateway

Nginx + Docker

Provides SSL termination, Gzip compression, upstream rate zones (5 req/min on login, 30 req/s on API), and DDoS shielding before reaching Uvicorn workers.

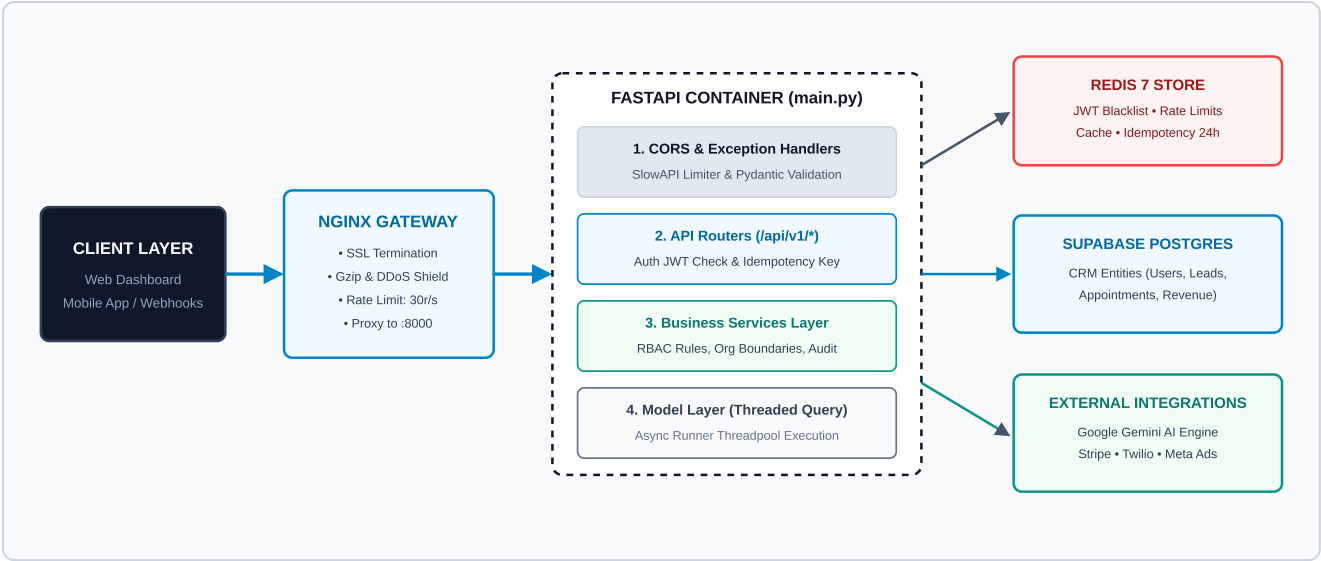
2. Layered Software Design Pattern

The codebase follows a strict multi-layer architectural contract to maintain separation of concerns:

Layer	Directory Path	Responsibility & Architectural Boundaries
API / Routers	backend/app/api/	Parses HTTP requests, handles rate limiting, extracts JWT credentials, validates input payloads via Pydantic, and delegates all work to services.
Service Layer	backend/app/services/	Contains 100% of core business logic, RBAC permission checks, tenant boundary enforcement, audit logging, and caching rules.
Data Models	backend/app/models/	Executes database interactions using Supabase Admin Client wrapped inside threadpools (<code>async_runner</code>) or SQLAlchemy ORM for AI tables.
Data Schemas	backend/app/schemas/	Strict Pydantic contracts defining API request inputs and JSON response shapes. Enforces structural validation before logic execution.
Core / Platform	backend/app/core/	Global platform infrastructure: system settings, Redis connection resiliency, JWT token decoding, exception handlers, and security roles.

3. End-to-End System Architecture & Request Lifecycle

Every incoming client request traverses a structured multi-stage processing pipeline before committing changes to the storage layer or returning data.



4. Multi-Tenant Boundary & Security Model

Multi-tenancy isolation is enforced at both the **Application Level (Services)** and the **Database Level (RLS Policies)** to prevent cross-tenant data leaks.

Organization Boundary

`app/services/`

Every CRM user is bound to an `organization_id`. Non-super admin users can only query, edit, or create entities belonging strictly to their tenant organization ID.

Clinic Access Enforcement

`app/core/roles.py`

Users (e.g., Clinic Managers, Agents, Receptionists) carry an array of `assigned_clinics`. Service operations enforce explicit checks ensuring target clinic operations fall within assigned IDs.

Standardized Error Envelope Contract

`app/core/exceptions.py`

All application exceptions (400, 401, 403, 404, 409, 422, 429, 500) return a strictly formatted JSON payload to allow deterministic client error parsing:

```
{ "success": false, "error": { "code": "FORBIDDEN_ACCESS", "message": "User lacks required permission" }
```

5. Directory Structure & File-by-File Breakdown

Below is the architectural file mapping of the repository (`backend/app/`), detailing the exact function and responsibility of each code module.

`backend/app/` |— `main.py` # Application entrypoint, route inclusion, middleware setup |— `database.py`

Comprehensive Module Responsibility Matrix

Module Path	Key Classes / Functions	Exact Responsibility & Workflow
<code>core/config.py</code>	<code>Settings</code>	Parses <code>.env</code> into typed variables (JWT keys, Supabase URLs, Redis URIs, AI models).
<code>core/roles.py</code>	<code>UserRole</code> , <code>has_permission()</code>	Catalog of 6 roles (<code>super_admin</code> to <code>finance</code>) and 50+ permissions.
<code>core/redis_client.py</code>	<code>ResilientRedisClient</code>	Tries Redis connection; seamlessly falls back to <code>InMemoryFallbackStore</code> if offline.
<code>core/idempotency.py</code>	<code>execute_idempotent()</code>	Prevents duplicate mutations on financial and booking endpoints via 24h response caching.
<code>services/auth.py</code>	<code>AuthService</code> , <code>get_current_user</code>	Handles login verification, HS256 JWT access/refresh token generation, and Redis revocation.
<code>services/openai_service.py</code>	<code>OpenAIService</code>	Wraps AsyncOpenAI client directed to Gemini endpoints with Tenacity retries and model fallback.
<code>services/webhook.py</code>	<code>WebhookService</code>	Verifies Stripe HMAC-SHA256, Twilio HMAC-SHA1, and Meta signatures with 7-day deduplication.
<code>core/async_runner.py</code>	<code>execute_query()</code>	Offloads blocking Supabase PostgREST sync calls to worker threads via <code>anyio.to_thread</code> .

6. Core Architectural Concepts Mapping (Part 1: Features 1–8)

Detailed analysis of how key backend concepts are implemented across the codebase:

1. Modular REST Endpoints <code>app/api/*.py</code> Concept: Over 50 REST endpoints organized cleanly into domain-specific routers. Implementation: Mounted under <code>/api/v1</code> prefix via <code>app.include_router()</code> in <code>main.py</code> across 11 feature routers.	2. HTTP Methods & RPC Actions <code>app/api/leads.py</code> Concept: Strict usage of GET, POST, PUT, PATCH, DELETE plus RPC command endpoints. Implementation: PATCH applied for partial updates; RPC endpoints (<code>/assign</code>, <code>/cancel</code>, <code>/checkin</code>) perform state transitions.
3. Pydantic Request/Response <code>app/schemas/</code> Concept: Typed data contracts ensuring invalid JSON never reaches services. Implementation: 15 schema modules. Action endpoints return unified <code>ActionSuccessResponse(success, message, data)</code>.	4. Standardized Status Codes <code>app/core/exceptions.py</code> Concept: Explicit HTTP codes (201 Created, 204 No Content, 400, 401, 403, 404, 422, 429). Implementation: Router response status decorators; global handlers format exception details into standard JSON error envelopes.
5. Auth & Token Blacklist <code>app/services/auth.py</code> Concept: Stateless JWT security with instant server-side logout revocation. Implementation: HTTPBearer header parser, bcrypt password hashing, Redis key <code>revoked:jti:{id}</code> check on every request.	6. Authorization (RBAC) & Tenants <code>app/core/roles.py</code> Concept: Fine-grained permissions and multi-clinic boundary enforcement. Implementation: 6 roles (Super Admin to Finance), 50+ permissions mapped via <code>has_permission()</code> and tenant boundary verification.
7. Access & Refresh Rotation <code>app/services/auth.py</code> Concept: Dual-token authentication with automatic refresh token rotation. Implementation: HS256 JWT 30-min access tokens and 7-day refresh tokens rotated via <code>POST /auth/refresh</code> with old token revocation.	8. Throttling & Rate Limiting <code>app/core/rate_limiter.py</code> Concept: Abuse protection at application and gateway levels. Implementation: SlowAPI limiter with Redis backend; <code>/auth/login</code> restricted to 5 req/min; AI routes limited to 10 req/min per clinic.

7. Core Architectural Concepts Mapping (Part 2: Features 9–16)

Continuation of architectural mappings covering integrations, caching, and infrastructure:

9. OAuth 2.0 (Google SSO)

`app/api/auth.py`

Concept: Third-party social sign-in mapped to internal user accounts.

Implementation: `/auth/oauth/google/url` generates stateful authorization URL; `/auth/oauth/google` validates ID token and issues JWTs.

10. Universal Pagination

`app/schemas/pagination.py`

Concept: Standardized query parameters and envelope for collection lists.

Implementation: Generic container

`PaginatedResponse[T]` returning `{data, total, page, limit, total_pages}` across all list endpoints.

11. Caching & Invalidation

`app/core/cache.py`

Concept: Accelerates heavy read calculations with automated write invalidation.

Implementation: Redis `CacheManager` caches reports & revenue totals; write operations trigger explicit key invalidation.

12. Financial Idempotency

`app/core/idempotency.py`

Concept: Prevents duplicate credit card charges or duplicate bookings on retries.

Implementation: `Idempotency-Key` header checked against Redis. Locks key during processing and replays cached response for 24 hours.

13. Webhooks Verification

`app/api/webhooks.py`

Concept: Secure processing of asynchronous external event notifications.

Implementation: Stripe HMAC-SHA256 signature check, Twilio form validation, Meta challenge verification, and 7-day event deduplication.

14. API Version Isolation

`app/core/config.py`

Concept: Prevents breaking contract changes for deployed frontends.

Implementation: Routers configured with `settings.api_prefix (/api/v1)`. Prepped for future `/v2` transitions with Deprecation headers.

15. OpenAI / Gemini Engine

`app/services/openai_service.py`

Concept: Resilient AI LLM integration for clinical lead summarization.

Implementation: `AsyncOpenAI` pointed to Gemini endpoints; Tenacity exponential backoff retries with model fallback and token cost logging.

16. API Gateway & Nginx Proxy

`deploy/nginx/nginx.conf`

Concept: Production edge security, SSL, rate shaping, and static compression.

Implementation: Docker Compose runs Nginx reverse proxy with Gzip, DDoS rate limit zones, and secure upstream proxying to Uvicorn.

8. Step-by-Step Request Execution Traces

Concrete trace walkthroughs showing exact execution order across files and components:

Trace A: Booking an Appointment with Idempotency & RBAC Check

- 1 Client Request:** Sends `POST /api/v1/appointments/` with Header `Authorization: Bearer <JWT>` and `Idempotency-Key: 7a8b-9c0d`.
- 2 Gateway & Limiter:** Nginx passes request to FastAPI. SlowAPI verifies IP rate limit (`app/core/rate_limiter.py`).
- 3 Authentication:** `get_current_user` dependency extracts JWT, decodes HS256 signature, verifies expiry, and checks Redis blacklist (`app/services/auth.py`).
- 4 Idempotency Check:** `execute_idempotent()` checks Redis key `idempotency:7a8b-9c0d`. If key exists, replays stored response instantly (`app/core/idempotency.py`).
- 5 RBAC & Tenant Check:** Service evaluates `has_permission(role, "appointment:create")` and verifies target `clinic_id` belongs to user's `assigned_clinics` (`app/services/appointment.py`).
- 6 Database Write & Audit:** `AppointmentModel.create()` inserts record via threadpool executor (`async_runner.py`). Write action logged to audit table (`app/services/audit.py`).
- 7 Cache & Return:** Response cached under idempotency key in Redis (24h). Returns HTTP 201 Created with `Appointment` JSON schema.

Trace B: AI Lead Summarization Flow with Retries & Token Audit

- 1 Client Request:** Staff user submits lead notes to `POST /api/v1/ai/summarize-lead` with header `X-Clinic-ID: clinic-123`.
- 2 Clinic Rate Limit:** `get_clinic_rate_limit_key()` resolves key `clinic-123` and enforces max 10 req/min limit via SlowAPI (`app/core/rate_limiter.py`).
- 3 OpenAI / Gemini SDK Call:** `OpenAIService.generate_lead_summary()` prepares system prompt and triggers `AsyncOpenAI` request (`app/services/openai_service.py`).
- 4 Resilient Retries & Fallback:** If primary model (`GEMINI_MODEL`) hits rate limits, Tenacity retries up to 3 times. If persistent, automatically falls back to `GEMINI_FALLBACK_MODEL`.
- 5 Cost Auditing:** Extracts prompt tokens, completion tokens, and calculates total cost. Inserts execution log into PostgreSQL table `ai_runs` (`app/models/ai_automation.py`).
- 6 Return Response:** Formats summary output into Pydantic schema and returns HTTP 200 OK to frontend.

9. Beginner's Technical Glossary & Mental Model

Non-technical analogies mapping complex backend terminology to daily real-world concepts:

Technical Term	Real-World Dental Clinic Analogy	Where it lives in Codebase
JWT Token	A plastic visitor wristband stamped with your name and role that lets you walk past clinic security.	<code>app/services/auth.py</code>
Token Blacklist	A "Lost Wristband" clipboard at reception. If a stolen wristband ID is on the board, security rejects entry.	Redis key <code>revoked:jti:{id}</code>
RBAC Permissions	Special keycards. Receptionists can open the waiting room door, but only surgeons can open operating rooms.	<code>app/core/roles.py</code>
Pydantic Schema	A strict paper intake form. If a patient leaves phone number blank, reception rejects the form immediately.	<code>app/schemas/*.py</code>
Idempotency Key	A receipt number on a payment form. If a patient submits the same form twice, the accountant ignores copy 2.	<code>app/core/idempotency.py</code>
Redis Cache	A quick sticky-note on the computer monitor showing daily revenue totals so you don't dig through filing cabinets.	<code>app/core/cache.py</code>
Reverse Proxy (Nginx)	A security guard standing outside the clinic front door directing traffic, checking IDs, and preventing floods.	<code>deploy/nginx/nginx.conf</code>
Rate Limiting	A ticket dispenser machine limiting guests to 5 questions per minute so nobody blocks the desk.	<code>app/core/rate_limiter.py</code>

10. Quick Developer Reference & File Lookup Index

I Want To... • Change token expiration time: Edit <code>ACCESS_TOKEN_EXPIRE_MINUTES</code> in <code>app/core/config.py</code>. • Add new permission to role: Update <code>ROLE_PERMISSIONS</code> dictionary in <code>app/core/roles.py</code>. • Modify appointment booking rules: Update <code>AppointmentService</code> in <code>app/services/appointment.py</code>.	I Want To... • Adjust login rate limits: Change SlowAPI decorator limit in <code>app/api/auth.py</code>. • Update AI model or prompt: Modify <code>GEMINI_MODEL</code> in <code>.env</code> or adjust prompts in <code>app/services/openai_service.py</code>. • Add new DB table: Create migration in <code>alembic/versions/</code> and add ORM model in <code>app/models/</code>.
--	--

Documentation Verification: This technical guide accurately reflects the complete architecture of the Dental CRM backend repository as of September 2026. All patterns described conform directly to production code.